HH Interval-Masked Streaming Top-k Pearson Correlation

Interval-Masked Streaming Top-k Pearson Correlation

Advisor: Hang, hang.hu@inf.ethz.ch

Problem Definition

Given a real-valued matrix X of shape (F, S) (F features, S samples), compute for every feature i the **top-k most correlated** other features $j \neq i$ using **Pearson correlation** [1], **but only over the pairwise valid overlap interval.** Input values are floating-point. The primary evaluation targets regimes with **large overlap lengths.**

We provide pseudo code for both algorithms and input generation as exaplined below.

Each feature i comes with a validity interval:

- $start_i$ (inclusive), end_i (exclusive), $0 \leq start_i < end_i \leq S$.

Per-feature z-score normalization (over its own valid interval):

- $\mu_i = mean(X[i, start_i : end_i])$
- $\sigma_i = std(X[i, start_i : end_i])$, use $\sigma_i = sqrt(var + eps)$ to avoid divide-by-zero
- $Xn[i, t] = (X[i, t] - \mu_i)/\sigma_i$ for $t \in [start_i, end_i)$.

Pairwise correlation over overlap:

For (i, j) define overlap:

- $L = max(start_i, start_j)$, $R = min(end_i, end_j)$.
- If $R - L < 2$: skip (or define correlation as 0).
- Else $corr(i, j) = (1/(R - L - 1)) * dot(Xn[i, L : R], Xn[j, L : R])$.

Output (online top-k):

For each i, output up to k pairs $(j, corr(i, j))$, sorted by decreasing $|corr|$.

Optional filter: keep only pairs with $|corr| \geq \tau$ (but still return up to k if enough exist).

The scratched pseudocode can be seen here [2] and can be used to create an unoptimized baseline.

What You Need To Do

1. **Baseline implementation**
   - Straightforward correct reference version.
   - Unit tests + correctness checks against a slow reference (e.g., double-precision baseline or Python reference for small sizes).
2. **High-performance implementation**

   Use the techniques taught in the course (code style, blocking, locality, ILP, profiling-driven optimization, SIMD, etc.)

   You are allowed to:
   - change data layout (transpose, tiling, padding, alignment),
   - rearrange computation order,
   - fuse stages,
   - design custom internal formats.
3. **Performance analysis**

   Show speedups and explain bottlenecks (profiling + roofline-style reasoning / bandwidth vs compute).

Rules

R1. Streaming Top-k (required)

The optimized version should **not materialize** the full correlation matrix $F \times F$. You should update top-k **online** while computing blocks (small temporary buffers allowed; no "store all then select").

R2. Adaptivity / auto-tuning (Optional)

The final implementation can adapt to different regimes (e.g., small vs large S, or short vs long average overlap).

Additional constraints

- BLAS or similar libraries may be used only for **sanity checks or comparison plots**, not as the main solution that computes all correlations.
- Focus on constant-factor performance; do not change the problem into an approximate method unless explicitly approved.

Benchmarking

Teams should provide a **deterministic generator** for synthetic inputs (we provide one scratched example here [3]). In your evaluation, you are encouraged to explore performance across multiple benchmark directions. For example, you can vary problem sizes/parameters, switching between different interval regimes, and using different data regimes such as block-correlated features so that top-k results are meaningful. You do **not** need to cover all combinations.

Report runtime and throughput metric that accounts for the effective work performed (e.g., proportional to the total overlap length processed). You can also briefly discuss peak memory usage.

---

[1] https://en.wikipedia.org/wiki/Pearson_correlation_coefficient

[2] https://polybox.ethz.ch/index.php/s/8GwGLmxyL2fEy3H

[3] https://polybox.ethz.ch/index.php/s/ofg4frbWksQpfBD